# SQLAnalyst — Project Documentation

Oct 10, 2026 · @innovative

SQLAnalyst lets anyone ask a database a question in plain English and get back the SQL, the result table, a chart and a short explanation — safely, read-only, with role-based access.

## 1. Requirements

**Problem.** Business users need answers that live in SQL databases, but most cannot write SQL. They wait on analysts, and giving them direct database access is risky. SQLAnalyst puts an AI assistant in between: it writes the SQL, checks that the SQL is safe, runs it read-only and explains the result.

### Users and roles

| Role | Who | What they can do |
|---|---|---|
| User | Business staff, analysts | Ask questions in chat, use the read-only SQL console, upload CSV/Excel files, see their own history |
| Admin | Team lead, DBA | Everything a user can, plus the admin panel: users, audit log, statistics, data sources, schema refresh, confirmed write changes |

### Functional requirements

| # | Requirement | Where it is built |
|---|---|---|
| F1 | Sign up with email and password, verified by a 6-digit emailed code | `auth.py`, `email.py` |
| F2 | Sign in with email/password or Google; forgot/reset password by email link | `auth.py`, `google_auth.py`, `email_tokens.py` |
| F3 | Ask a question in plain English and get SQL, a result table, a chart and an explanation, streamed live | `ask_stream.py`, `query_pipeline.py`, `prompts.py` |
| F4 | Choose which database to ask (default database or admin-added ones) | `sources.py`, `Navbar.tsx` |

| # | Requirement | Where it is built |
|---|---|---|
| F5 | See the database schema: tables, columns, keys and a relationship diagram | `sources.py`, `Sidebar.tsx`, `SchemaDiagram.tsx` |
| F6 | Remember earlier answers in the same conversation | `rag_memory.py` |
| F7 | Run typed SQL in a console with the same safety checks | `main.py` (`/api/query`), `sql/page.tsx` |
| F8 | Upload a CSV or Excel file and ask questions about it | `file_analysis.py`, `file_session_store.py` |
| F9 | Keep chat history per user; clear it | `chat_history.py` |
| F10 | Admin: manage users, view stats and a full audit log | `admin.py`, `admin/page.tsx` |
| F11 | Admin: add, test, edit and remove external databases | `data_sources.py`, `connection_manager.py` |
| F12 | Admin: make data changes (INSERT/UPDATE/DELETE/DDL) only after confirming | `admin_sql.py`, `admin_writes.py` |
| F13 | Explain joins visually (Venn circles) in the result | `sqlJoins.ts`, `JoinVenn.tsx` |

### Non-functional requirements

| Area | Requirement | How it is met |
|---|---|---|
| Safety | AI or user SQL must never change or leak data | 11-check validator, read-only transaction always rolled back, row cap, timeout |
| Security | Credentials and sessions protected | bcrypt passwords, short-lived JWT + httpOnly refresh cookie, token revocation, rate limits, encrypted data-source passwords |
| Performance | First result visible quickly | Server-Sent Events stream the table before the explanation; schema cached; optional Redis cache |

| Area | Requirement | How it is met |
|---|---|---|
| Portability | Work with common databases | SQLAlchemy + sqlglot dialects: PostgreSQL, MySQL, SQLite, SQL Server, Oracle |
| Usability | Usable by non-technical staff on any screen size | Plain-language answers, automatic charts, responsive Tailwind UI, animations that respect reduced-motion |
| Auditability | Every question traceable | `audit_logs` table records user, SQL, rows, time and errors |
| Maintainability | Easy to test and change | One shared query pipeline, typed models (Pydantic, TypeScript), pytest suite |

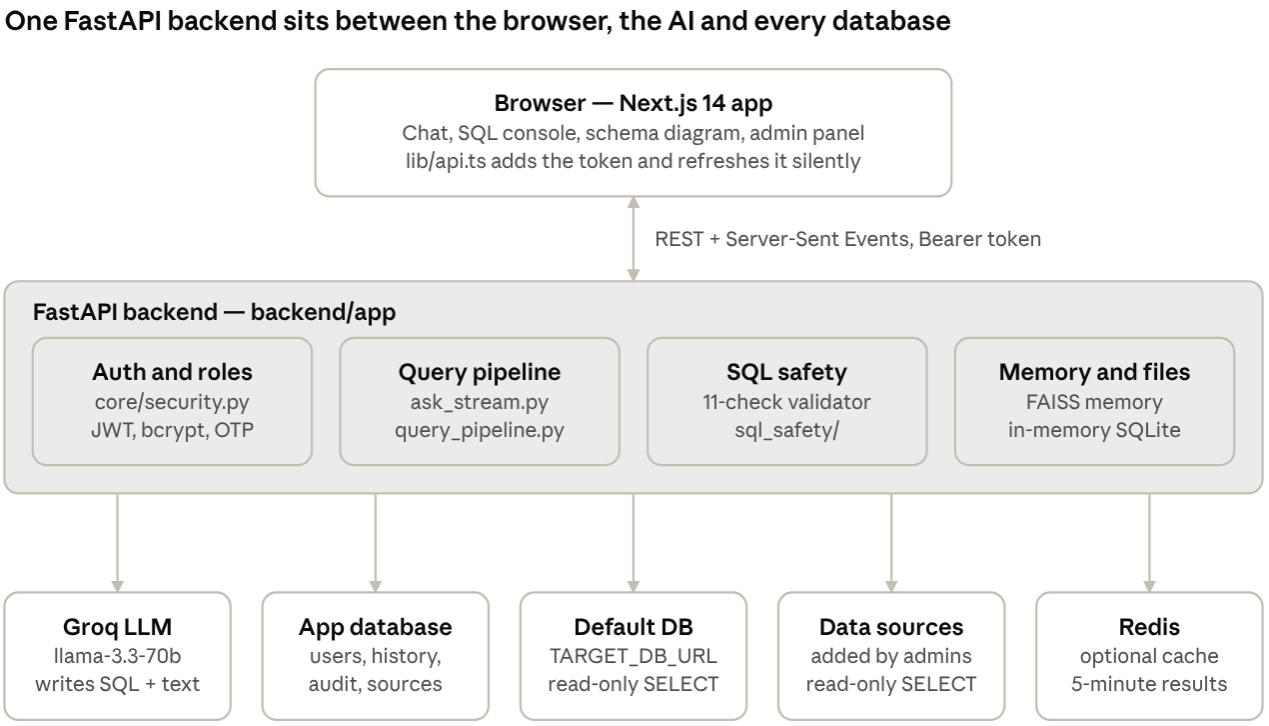
## 2. Tech stack

The project is a Python FastAPI backend and a TypeScript Next.js frontend, with Groq's hosted Llama model writing the SQL.

| Layer | Technology (version) | Why it was chosen |
|---|---|---|
| Frontend framework | Next.js 14.2 (App Router), React 18, TypeScript 5 | File-based routing, typed components, fast dev server |
| Styling and motion | Tailwind CSS 3.4, Framer Motion | Utility classes keep styles next to markup; smooth, consistent animations |
| Charts | Chart.js 4 + react-chartjs-2 | Simple bar/line/pie charts from result rows |
| Backend framework | FastAPI 0.141, Uvicorn | Async, automatic request validation and OpenAPI docs at `/docs` |
| Validation | Pydantic 2 | Request/response models with clear errors |
| ORM / DB access | SQLAlchemy 2 (async), aiosqlite, asyncpg, psycopg2, pymysql, aiomysql, aioodbc, oracledb | One API for every supported database |

| Layer | Technology (version) | Why it was chosen |
|---|---|---|
| LLM | Groq SDK, `llama-3.3-70b-versatile` | Very fast inference, good SQL quality, streaming |
| SQL parsing | sqlglot 30 | Parses SQL in each dialect so safety checks work on the syntax tree, not on text guesses |
| Conversation memory | sentence-transformers + FAISS | Finds the earlier Q&A most similar to the new question |
| File analysis | pandas, openpyxl, xlrd | Reads CSV/XLSX/XLS into an in-memory SQLite table |
| Auth | PyJWT, bcrypt, google-auth, slowapi | Tokens, password hashing, Google sign-in, rate limits |
| Encryption | cryptography (Fernet) | Encrypts saved data-source passwords |
| Cache | Redis 7 (optional) | Caches admin data-source query results |
| App database | SQLite (dev) / PostgreSQL 15 (Docker) | Stores users, codes, history, audit log, data sources |
| Tests and quality | pytest, pytest-asyncio, ruff, mypy, ESLint, Prettier | Automated tests and consistent code style |
| Deployment | Docker Compose | One command starts PostgreSQL, Redis, backend and frontend |

# 3. Architecture at a glance




System architecture · browser, backend modules, AI and databases

The browser only talks to the FastAPI backend. Inside it, auth checks every request, the query pipeline asks Groq for SQL, and the SQL safety module must approve that SQL before any database sees it. The app database stores SQLAnalyst's own records; the default database and admin-added data sources are only read.

# 4. Project structure

The repository has two apps — `backend/` (Python) and `frontend/` (TypeScript) — plus Docker and documentation files at the root.

### Root

| Path | Purpose |
|---|---|
| `docker-compose.yml` | Starts PostgreSQL 15, Redis 7, backend (:8000) and frontend (:3000) together |
| `docs/` | Markdown guides: setup, configuration, API, architecture, security, data sources |
| `check_errors.bat` | Windows script that runs every test, type check and lint, writing `check_results.txt` |

| Path | Purpose |
|---|---|
| `mypy.ini` | Type-checking settings for the backend |

### Backend — `backend/app/`

| Folder / file | What it holds |
|---|---|
| `main.py` | Creates the FastAPI app, runs start-up tasks, registers every router, `/health` and `/api/query` |
| `config.py` | `Settings` class: every environment variable with its default; production safety checks |
| `api/routes/` | One file per feature area of HTTP endpoints (auth, ask, sources, admin …) |
| `core/` | Cross-cutting helpers: `security.py` (JWT, passwords, current user), `email.py`, `email_tokens.py`, `rate_limit.py`, `avatar.py`, `friendly_errors.py` |
| `db/` | Default-database engine (`connection.py`), safe executor, schema cache (`schema_registry.py`), key/constraint reader (`table_rules.py`), app-DB session |
| `langchain_chains/` | LLM prompts (`prompts.py`) and conversation memory (`rag_memory.py`) |
| `models/` | SQLAlchemy tables: `User`, `OTPCode`, `ChatMessage`, `AuditLog`, `DataSource` |
| `schemas/` | Pydantic request/response shapes for auth, admin and data sources |
| `services/` | Business logic shared by routes: `query_pipeline.py`, `connection_manager.py`, `file_session_store.py`, `query_cache.py`, `sql_dialect.py` |
| `sql_safety/` | The read-only validator (`validator.py`, `rules.py`, `sanitizer.py`, `whitelist.py`) and the admin write checker (`admin_writes.py`) |
| `utils/logger.py` | One shared logger with a timestamped format |

### Backend — other files

| Path | Purpose |
|---|---|
| `tests/` | 7 pytest files: auth flow, SQL validator, sources, data sources, admin SQL, chat changes, file search |
| `data/` | `app.db` (app database in dev), `analytics.db` (sample shop database), `schema_cache.json` |
| `requirements.txt` | Pinned Python packages |
| `Dockerfile` | Python 3.13 image with ODBC drivers for SQL Server |
| `cleanup_users.py`, `test_smtp.py` | Maintenance scripts: remove test users, check email settings |

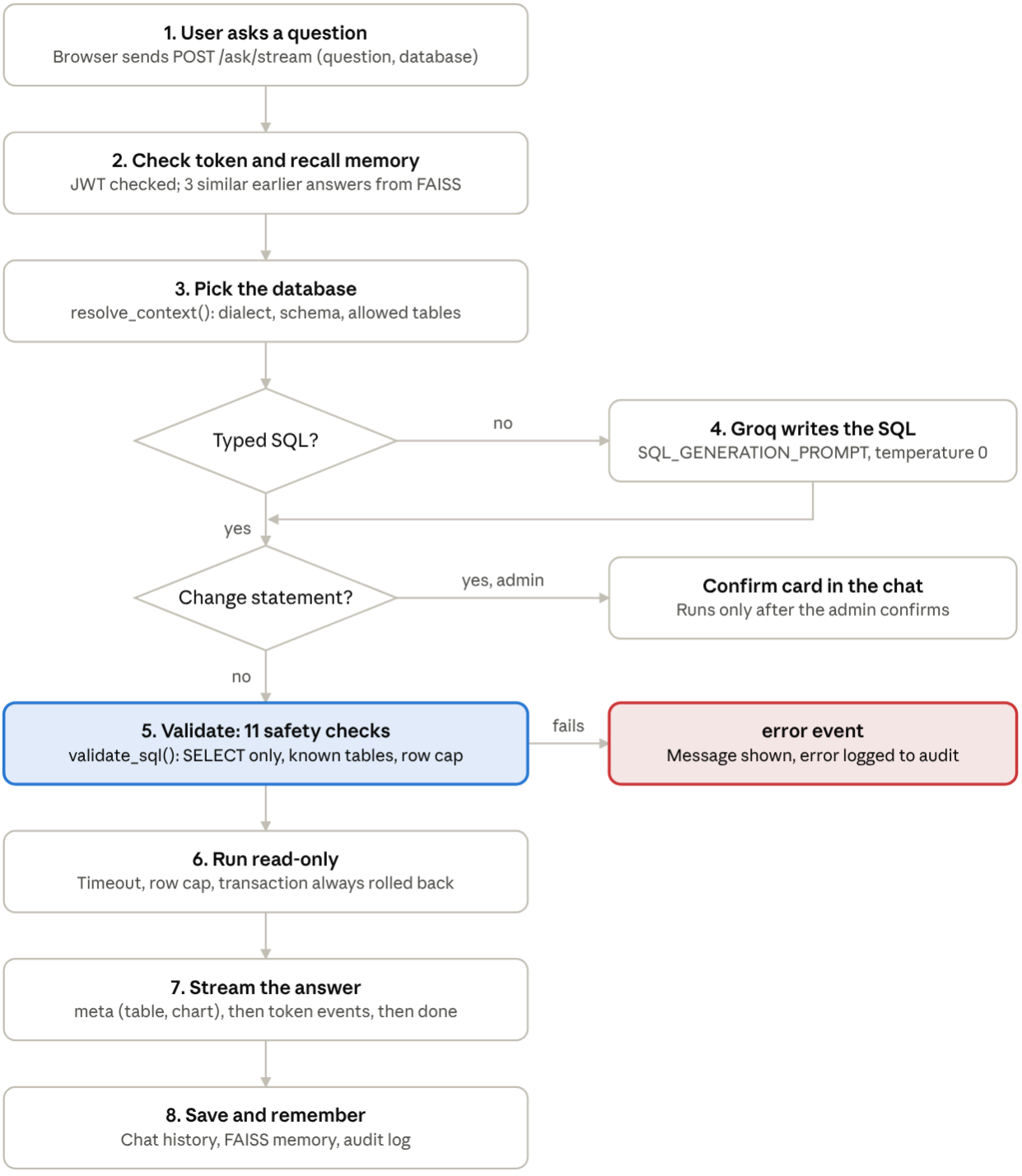
### Frontend — `frontend/`

| Path | What it holds |
|---|---|
| `app/layout.tsx`, `app/page.tsx` | Root HTML shell, fonts, motion provider; `/` redirects to the dashboard or login |
| `app/auth/` | Sign in, register, code verification, password reset; `google-success/` finishes Google login |
| `app/dashboard/layout.tsx` | Logged-in shell: auth guard, loads databases, schema, history; renders Navbar, Sidebar and the schema diagram |
| `app/dashboard/page.tsx` | The chat: streaming answers, chart/table/joins/SQL tabs, file upload, admin change cards |
| `app/dashboard/sql/page.tsx` | SQL console (read-only, plus admin write mode) |
| `app/dashboard/context.tsx` | React context shared by dashboard pages (selected database, suggestions) |
| `app/admin/` | Admin panel: overview stats, users, audit log, data sources |
| `components/` | Reusable UI: `Navbar`, `Sidebar`, `SchemaDiagram`, `JoinVenn`, `ChartBlock`, `DataTable`, `Avatar`, `Icon`, `LoadingScreen`, `motion` |
| `lib/api.ts` | The API client: tokens, silent refresh, typed calls to every endpoint |

| Path | What it holds |
| --- | --- |
| `lib/sqlJoins.ts` | Finds JOINs in a SQL string for the Venn view |
| `tailwind.config.ts`, `app/globals.css` | Theme colours, animations, shared classes (`.card`, `.btn-primary` …) |

# 5. Main workflow

**Every question passes the safety check before it touches a database**




Question-to-answer workflow · 8 steps, 2 decisions, 1 failure exit

A question goes down the left column; the right column holds the side paths. Typed SQL skips the AI, an admin's change stops at a confirm card, and a failed safety check ends with an error instead of reaching the database. Section 9 explains each step's code.

# 6. Backend startup and configuration

On start, the backend checks its settings, creates missing tables, reads the default database's schema, warms the AI memory model and makes sure an admin account exists.

**`config.py`**

| Function / class | What it does |
|---|---|
| `_int(name, default)`, `_float(name, default)` | Read a number from an environment variable; fall back to the default if missing or invalid |
| `Settings` | One attribute per setting, read from `backend/.env` (loaded with python-dotenv). Also exposes `is_production`, `is_development`, `cors_origins`, `cookie_secure` and `smtp_configured()` |
| `validate_production_settings(s)` | When `APP_ENV=production`, refuses to start if `SECRET_KEY` is weak or shorter than 32 characters, `DB_ENCRYPTION_KEY` is missing, or the admin password is still `Admin1234` |
| `get_settings()` | Builds `Settings` once (`lru_cache`); `settings` is the shared instance every module imports |

### Key settings

| Variable | Default | Meaning |
|---|---|---|
| `APP_ENV` | development | `production` turns on strict checks and hides error details |
| `SECRET_KEY` | change-me | Signs JWT tokens |
| `DATABASE_URL` | SQLite `data/app.db` | App database (users, history, audit) |
| `TARGET_DB_URL` | SQLite `data/analytics.db` | Default database users ask about |
| `GROQ_API_KEY`, `LLM_MODEL` | —, llama-3.3-70b-versatile | AI provider and model |

| Variable | Default | Meaning |
|---|---|---|
| `LLM_TEMPERATURE` / `LLM_MAX_TOKENS` | 0.0 / 500 | SQL generation: deterministic, short |
| `LLM_EXPLANATION_TEMPERATURE` / `_MAX_TOKENS` | 0.3 / 300 | Explanations: a little more natural |
| `DB_MAX_ROWS` | 10000 | Highest row cap any query can return |
| `DB_ENCRYPTION_KEY` | — | Fernet key for data-source passwords |
| `ACCESS_TOKEN_EXPIRE_MINUTES` / `REFRESH_TOKEN_EXPIRE_DAYS` | 15 / 7 | Token lifetimes |
| `PASSWORD_RESET_EXPIRE_MINUTES` | 60 | Reset-link lifetime |
| `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM` | port 465 | Outgoing email |
| `DEFAULT_ADMIN_EMAIL` / `_PASSWORD` / `_NAME` | admin@sqlanalyst.com / Admin1234 / Admin | First admin account |
| `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` | — | Google sign-in |
| `ADMIN_SQL_WRITES` | true | Lets admins run confirmed data changes |
| `REDIS_URL` | redis://localhost:6379 | Optional cache |
| `RATE_LIMIT_ENABLED` | true | Turns rate limits on or off |

### `main.py` — what runs, in order

1. `validate_production_settings(settings)` — runs on import, before anything else.
2. `lifespan(app)` — FastAPI's start-up hook:
   a. `Base.metadata.create_all` creates any missing app tables.
   b. `_sync_columns` adds columns introduced after the first release (listed in `_COLUMN_MIGRATIONS`: `users.avatar_url`, `users.token_version`, `otp_codes.attempts`).
   c. `_fix_sqlite_audit_ids` rebuilds an old SQLite `audit_logs` table whose `BIGINT` id could not auto-number.

   d. `query_cache.initialize()` connects to Redis if it is available.
   e. `refresh_schema()` reads the default database's tables into `schema_cache.json` (in a thread, as SQLAlchemy inspection is synchronous).
   f. `rag_warmup()` loads the sentence-embedding model so the first question is not 3–5 s slower.
   g. `_seed_default_admin()` creates the admin account if none exists, warning if the default password is kept.
3. App setup — `install_rate_limiter(app)`, CORS middleware for `CORS_ORIGINS` + `FRONTEND_URL` with credentials allowed, then `include_router` for the 11 route modules.
4. `health_check()` — `GET /health` runs `SELECT 1` (or `SELECT 1 FROM dual` on Oracle) on the default database and returns `healthy` or `degraded` with the server version.
5. `run_query()` — `POST /api/query` for the SQL console: loads the chosen data source, gets its dialect and tables with `resolve_context()`, validates with `validate_sql()`, runs with `run_pipeline_query()` and returns columns, rows, row count, truncation flag and time.

**Small shared helpers**

| File | Function | What it does |
|---|---|---|
| `core/rate_limit.py` | `limiter`, `install_rate_limiter(app)` | A slowapi limiter keyed by client IP; attaches it and the 429 handler to the app |
| `utils/logger.py` | `setup_logger(name)`, `logger` | One stderr logger with `time - level - name - message` format |

# 7. Authentication and roles

A user proves their email with a 6-digit code, then receives a 15-minute access token and a 7-day refresh cookie; every protected endpoint checks the token and the user's role.

**Sign-up and login flow**

1. **Register** — `POST /auth/register` with name, email and password. Pydantic checks the name (2+ characters) and password (8+ characters, one uppercase letter, one digit). The password is hashed with bcrypt and the account is saved with `email_verified = false`.

2. **Code emailed** — `_issue_otp()` deletes any old code for that email and stores a new one from `secrets.randbelow` (valid 10 minutes). `send_otp_email()` sends it.
3. **Verify** — `POST /auth/verify-otp` compares the code with `hmac.compare_digest` (constant-time). Each wrong try increases `attempts`; after 5 the code is burned. On success the email is marked verified and the user is logged in straight away.
4. **Login** — `POST /auth/login` checks the bcrypt hash, that the account is active and verified, then calls `issue_token_pair()`. Typing just `admin` maps to `DEFAULT_ADMIN_EMAIL`.
5. **Use the app** — the frontend sends `Authorization: Bearer <access token>` on every call.
6. **Silent refresh** — when the access token expires, the frontend calls `POST /auth/refresh`; the browser sends the httpOnly refresh cookie automatically and a new pair comes back.
7. **Logout** — `POST /auth/logout` increases the user's `token_version`, which makes every token issued before it invalid on every device, and clears the cookie.

**Password reset**

1. `POST /auth/forgot-password` always answers the same message, so nobody can test which emails exist.
2. For a real user it creates a reset JWT with `create_email_token()`, signed with a separate secret and carrying `password_fingerprint()` — the first 16 hex characters of a SHA-256 of the current password hash.
3. `POST /auth/reset-password` checks the token, then checks the fingerprint still matches. Once the password changes, the fingerprint changes too, so the link works only once. The reset also bumps `token_version`, logging the user out everywhere.

**Google sign-in** ( `google_auth.py` )

| Function / endpoint | What it does |
|---|---|
| `POST /auth/google/token` — `google_token_login()` | Popup flow: verifies Google's ID token with `google-auth`, rejects unverified Google emails, then logs the user in |
| `GET /auth/google/redirect` — `google_redirect()` | Redirect flow: sends the browser to Google's consent screen. If Google is not configured it refuses, except in development, where it logs in a test user |

| Function / endpoint | What it does |
|---|---|
| `GET /auth/google/callback` — `google_callback()` | Google returns here with a code; it is exchanged for the user's profile over HTTPS with `httpx` |
| `_get_or_create_google_user()` | Finds the user by email or creates one (random password, already verified), and saves their Google photo |
| `_success_redirect(user)` | Sends the browser to `/auth/google-success#token=…`. The token is in the URL fragment, which browsers never send to servers |

### `core/security.py` — the functions every route relies on

| Function | What it does |
|---|---|
| `hash_password(pw)` / `verify_password(pw, hash)` | bcrypt with cost factor 12 |
| `create_access_token(data, ver)` | JWT (HS256) with `type: access`, `ver`, 15-minute expiry |
| `create_refresh_token(data, ver)` | JWT with `type: refresh`, a unique `jti` and 7-day expiry |
| `decode_token(token)` | Verifies signature and expiry |
| `set_refresh_cookie()` / `clear_refresh_cookie()` | httpOnly, `SameSite=Lax`, `Secure` in production |
| `issue_token_pair(user, response)` | Makes both tokens, sets the cookie, returns the access token |
| `user_from_access_token(token, db)` | Decodes the token, loads the user, rejects wrong type, wrong `ver` (revoked) or a deactivated account |
| `authenticate_request(request)` | Same check for streaming routes, which open their own DB session |
| `get_current_user` | FastAPI dependency: any logged-in user |
| `require_admin` | FastAPI dependency: returns 403 unless `role == "admin"` |

### Other auth helpers

| File | Function | What it does |
|---|---|---|
| `core/email.py` | `_send()` | Sends HTML mail over SMTP: SSL on port 465, STARTTLS on other ports |
| | `send_otp_email()`, `send_password_reset_email()` | Build the branded emails; return `False` instead of crashing when mail fails |
| | `explain_smtp_error()` | Turns SMTP errors into a fix an admin can act on (e.g. wrong app password) |
| `core/email_tokens.py` | `create_email_token()`, `verify_email_token()`, `password_fingerprint()` | Single-use reset tokens (above) |
| `core/avatar.py` | `resolve_avatar(user)` | Google photo if saved, otherwise a Gravatar URL for the email |
| `routes/auth.py` | `_dev_only()` | In development only, returns the code or reset link in the response when email fails |
| | `GET /auth/me` — `me()` | Returns the logged-in user's profile |

### Data stored (`models/user.py`, `models/otp.py`)

| Table | Main columns |
|---|---|
| `users` | `id` (UUID), `email` (unique), `name`, `password_hash`, `role` (`user` / `admin`), `is_active`, `email_verified`, `avatar_url`, `token_version`, `created_at`, `updated_at` |

| Table | Main columns |
|---|---|
| `otp_codes` | `email`, `code` (6 digits), `used`, `attempts`, `expires_at`; `is_expired()` compares with the current UTC time |

**Rate limits** (per IP): register, send-otp and forgot-password 5/minute; login, verify-otp, reset-password and Google token 10/minute; refresh 30/minute. Going over returns `429 Too Many Requests`.

## 8. Database layer

The backend talks to two kinds of database: the **app database** (`DATABASE_URL`, async) that stores the application's own records, and the **databases users ask about** (`TARGET_DB_URL` plus admin-added data sources), which are only ever read.

**App database tables** (`models/`)

| Table | Model | Main columns | Links |
|---|---|---|---|
| `users` | `User` | id, email, name, password_hash, role, is_active, email_verified, avatar_url, token_version | — |
| `otp_codes` | `OTPCode` | email, code, used, attempts, expires_at | by email |
| `chat_messages` | `ChatMessage` | user_id, question, sql_query, explanation, engine_used, session_id, created_at | user_id → users (delete cascades) |
| `audit_logs` | `AuditLog` | user_id, session_id, question, sql_used, row_count, status, error_msg, timing_ms, created_at | user_id → users (set null on delete) |
| `data_sources` | `DataSource` | name, db_type, host, port, username, database_name, encrypted_password, is_active | user_id → users (the admin who added it) |

`AuditLog.id` is `BIGINT` on PostgreSQL/MySQL but `INTEGER` on SQLite, because only `INTEGER PRIMARY KEY` auto-numbers in SQLite.

**Files in `db/`**

| File | Function / object | What it does |
|---|---|---|
| `base.py` | `Base` | SQLAlchemy `DeclarativeBase` every model inherits from |
| `session.py` | `engine`, `AsyncSessionLocal` | Async engine for the app database (pool of 10 + 20 overflow on PostgreSQL; plain on SQLite) |
| | `get_db()` | FastAPI dependency: opens a session per request, rolls back on error, always closes |
| `connection.py` | `_build_engine()` | Builds a synchronous engine for `TARGET_DB_URL` with `pool_pre_ping` (pool 5 + 10, not for SQLite) |
| | `get_engine()` | Returns the cached engine, building it on first use |
| | `reset_engine()` | Disposes the engine so the next call reconnects — used by schema refresh |
| | `get_target_connection()` | Context manager giving a raw DB-API connection that is always closed |
| | `_EngineProxy`, `engine` | Lets old `from app.db.connection import engine` imports always reach the live engine |
| `executor.py` | `QueryResult` | Dataclass: columns, rows, row_count, timing_ms, truncated |
| | `execute_safe_query(sql, max_rows)` | Runs already-validated SQL on the default database: sets a 5-second timeout (PostgreSQL/MySQL), fetches at most `cap + 1` rows to detect truncation, and **always rolls back** in `finally` |

| File | Function / object | What it does |
|---|---|---|
| `schema_registry.py` | `_introspect_db()` | Uses SQLAlchemy `inspect()` to list every table and column, counts rows (trying back-tick, double-quote and plain quoting) and adds key rules |
| | `_save_cache()`, `_load_cache()` | Write/read `data/schema_cache.json` |
| | `_cached_schema()` | `lru_cache` so the file is read once per process |
| | `refresh_schema()` | Reset engine → introspect → save only if tables came back (a network blip cannot wipe a good cache) → clear the cache |
| | `get_schema()`, `get_allowed_tables()` | Cached schema and the lower-case set of table names the validator allows |
| `table_rules.py` | `table_rules(insp, table)` | Reads primary key, foreign keys (as `col → table(col)`), unique and CHECK constraints. The AI uses them to write valid changes; the schema diagram uses the keys to draw relationships |

**Legacy schema endpoints** (`routes/schema.py`): `GET /api/schema` returns the cached tables and suggestions; `POST /api/schema/refresh` (admin) re-reads the default database. The frontend now uses the newer `/sources/{id}/schema`, which works for every database.

## 9. Natural language to SQL pipeline

The AI never touches a database directly: it only writes text, and that text must pass an 11-check validator before it runs in a transaction that is always rolled back.

### 9.1 The streaming endpoint — `routes/ask_stream.py`

`POST /ask/stream` receives `{question, session_id, data_source_id}` and answers with Server-Sent Events. `ask_stream()` checks the token first (so a bad token gets a clean 401, not a broken stream), then hands over to the generator `_stream_response()`:

| Step | Code | What happens |
|---|---|---|
| 0. Recall | `get_session_rag(f"{user_id}:{session_id}").retrieve(question, 3)` | Up to 3 similar earlier Q&A pairs from this user's conversation |
| 1. Pick the database | `load_active_data_source()`, `resolve_context()` | Returns the SQL dialect, schema and allowed tables of the chosen database |
| 2. Get SQL | `is_direct_sql()` or `generate_sql()` | Typed SQL is used as is; otherwise Groq writes it |
| 2b. Admin change? | `is_write_sql()`, `validate_admin_sql()` | A write statement is never run here. Admins get a `confirm` event with the SQL; everyone else gets an error |
| 3. Validate | `validate_sql(sql, dialect, allowed_tables)` | The 11 checks (9.4) |
| 4. Run | `run_query(sql, data_source)` | Read-only, row-capped, timed out, rolled back |
| 5. `meta` event | `infer_chart()` | SQL, columns, rows, row count, chart type, time — the table and chart appear now |
| 6. `token` events | `_stream_explanation()` | Groq streams a short business explanation, chunk by chunk |
| 7. Fallback | `fallback_explanation()` | Plain summary if the AI is not configured or returned nothing |
| 8. Remember and log | `_fire_and_forget(rag.add(...))`, `_write_audit_log()` | Saved in the background so the user is not kept waiting |
| 9. `done` event | `_sse("done", …)` | Full explanation; the frontend then saves the turn to chat history |

Any failure sends an `error` event and writes an `error` row to the audit log.

Helpers in the same file: `_sse(event, data)` formats one SSE frame (`event: …\ndata: …\n\n`); `_stream_explanation()` runs Groq's blocking stream in a worker thread and passes tokens back through an `asyncio.Queue`, so the server never blocks; `_fire_and_forget()` keeps a reference to background tasks so Python does not garbage-collect them early.

`POST /ask` (`routes/ask.py`, `ask_question()`) runs the same steps but returns one JSON response instead of a stream.

## 9.2 Prompts — `langchain_chains/prompts.py`

| Prompt | Used for | Key rules given to the model |
|---|---|---|
| `SQL_GENERATION_PROMPT` | Question → SQL (system message) | Target dialect by name; the schema as JSON; use only listed tables/columns; output raw SQL only; no comments or semicolons; alias aggregates; limit rows; put the label column first and the number second so charts render |
| `READ_ONLY_RULES` | Inserted for everyone | Exactly one SELECT/WITH; never INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE |
| `ADMIN_WRITE_RULES` | Inserted for admins when write mode is on | One write statement only when explicitly asked; WHERE on UPDATE/DELETE; obey CHECK values exactly, defaults, NOT NULL and foreign keys (look up ids with a subquery) |
| `RAG_EXPLANATION_PROMPT` | Result → explanation | Earlier conversation, question, SQL and the first 20 rows; explain business meaning, not SQL; under 4 sentences |

SQL generation uses temperature 0.0 (same question, same SQL). Explanations use 0.3 for more natural wording.

## 9.3 Shared logic — `services/query_pipeline.py`

| Function | What it does |
|---|---|
| `llm_configured()` | True when `GROQ_API_KEY` looks real (starts with `gsk_`) |
| `generate_sql(question, schema, dialect, allow_writes)` | Builds the prompt, calls Groq, cleans the answer, and raises `SQLGenerationError` if the model refused or did not return SELECT/WITH (or an allowed write for admins) |
| `clean_sql_output(raw)` | Removes markdown code fences and trailing semicolons |
| `is_direct_sql(question, …)` | Decides whether the user typed SQL. The first word is not enough: it parses the text with sqlglot, so “delete order id 92” or “select the top customers” go to the AI as sentences |
| `_parses_as_sql()` | The parser test behind `is_direct_sql()`, including a check that named tables really exist |
| `is_write_sql(sql)` | True for INSERT, UPDATE, DELETE, MERGE, CREATE, DROP, ALTER, TRUNCATE |
| `load_active_data_source(db, id)` | Loads an active data source or raises 400/404 |
| `load_data_source_schema(ds)` | Reads its tables, with a clear 502 if the connection fails and 400 if there are no tables |
| `resolve_context(ds)` | `(dialect, schema, allowed_tables)` for a data source or the default database |
| `run_on_data_source(ds, sql)` | Runs on an admin-added database: 30-second timeout, `cap + 1` fetch, always `rollback()` |
| `run_query(sql, ds)` | Chooses `run_on_data_source()` or `execute_safe_query()` (in a thread) |
| `infer_chart(columns, rows)` | No chart for one row or no numbers; **line** if the label looks like a date; **pie** for 6 or fewer non-negative values; otherwise **bar** |

| Function | What it does |
|---|---|
| `fallback_explanation()` | A sentence such as “Returned 5 rows. The first entry is …” when the AI is unavailable |
| `public_error(message, exc)` | Uses `core/friendly_errors.py` to turn driver errors into one plain sentence; driver details only outside production |
| `source_label(ds)` | Readable database name for messages |

`services/sql_dialect.py` maps each database type to a sqlglot dialect (`postgresql→postgres`, `sqlserver→tsql`, …) with `dialect_for_db_type()`, `dialect_from_url()` and `human_dialect_name()`.

## 9.4 The 11-check SQL validator — `sql_safety/`

`validate_sql(raw_sql, dialect, allowed_tables, max_rows)` returns safe SQL or raises `SQLValidationError(check, message)`. One trailing semicolon is removed first.

| # | Check | Where |
|---|---|---|
| 1 | No `--` or `/* */` comments | `rules.check_no_comments` |
| 2 | No semicolons (stops statement chaining) | `rules.check_no_semicolons` |
| 3 | Starts with SELECT or WITH | `rules.check_starts_with_select` |
| 4 | No blocked keyword (40 words: DROP, DELETE, INSERT, INTO, GRANT, EXEC, ATTACH, PRAGMA, COPY, SLEEP, PG_READ_FILE, XP_CMDSHELL …) | `rules.check_no_blocked_keywords`, `whitelist.BLOCKED_KEYWORDS` |
| 5 | Parses cleanly with sqlglot in the target dialect | `validator.py` |
| 6 | Exactly one statement | `validator.py` |
| 7 | Root is SELECT or a set operation (UNION) | `validator.py` |
| 8 | No Insert/Update/Delete/Drop/Create/Alter/Merge/Into/Command node anywhere in the syntax tree | `validator.py` (`stmt.walk()`) |
| 9 | No system schemas or tables (`information_schema`, `pg_catalog`, `sqlite_master`, `mysql`, `sys` …) | `validator.py` + `rules.check_no_system_tables` |
| 10 | Every table is one of that database's tables (CTE names allowed) | `validator.py` |
| 11 | Row cap: `LIMIT` added or lowered; `TOP` / `FETCH FIRST` for SQL Server/Oracle via an AST rewrite | `_enforce_limit()`, fallback `sanitizer.inject_limit()` |

Checks 1–4 run on the SQL with string values blanked out ( `strip_string_literals()` ), so `WHERE status = 'open -- now'` is not mistaken for a comment. Keyword matching uses word boundaries, so a column called `created_at` does not trip `CREATE` .

### 9.5 Conversation memory — `langchain_chains/rag_memory.py`

| Part | What it does |
|---|---|
| `EMBED_MODEL_NAME = "all-MiniLM-L6-v2"` | Small, fast sentence-embedding model (384 numbers per text) |
| `_load_libs()` , `_get_embed_model()` , `warmup()` | Load FAISS and the model lazily; `warmup()` runs at start-up |
| `_embed(texts)` | Turns text into normalised vectors |
| `SessionRAG.add(q, a)` | Stores `Q: … A: …` and adds its vector to a FAISS inner-product index (cosine similarity) |
| `SessionRAG.retrieve(query, top_k=3)` | Returns the most similar stored turns |
| `get_session_rag(key)` | One store per `user_id:session_id` , at most 500 stores (oldest removed). Keying by user means nobody can read another person's memory by guessing a session id |

Memory is held in RAM, so it resets when the server restarts; saved chat history in the database does not.

## 10. Data sources, file analysis and admin features

Beyond the default database, admins can connect other databases, anyone can upload a spreadsheet, and admins can make confirmed data changes.

### 10.1 Data sources — `routes/data_sources.py` , `services/connection_manager.py`

An admin adds a PostgreSQL, MySQL, SQLite, SQL Server or Oracle database; every user can then pick it in the top-bar switcher.

| Function | What it does |
| --- | --- |
| `create_data_source()` — `POST /data-sources` | Requires host, port, username and database name (only the file path for SQLite), encrypts the password, saves the row |
| `list_data_sources()`, `get_data_source()` | List or read one (password never returned — `_to_response()`) |
| `update_data_source()` — `PATCH` | Edits fields, re-encrypts a new password, and clears cached engine, schema and query results |
| `delete_data_source()` — `DELETE` | Removes it and its caches |
| `test_data_source_connection()` — `POST /data-sources/test` | Builds a fresh engine for unsaved details and runs `SELECT 1` (`SELECT 1 FROM dual` on Oracle); reports success and time |
| `get_data_source_schema()` | Reflects tables with their row counts for the admin screen |
| `execute_query_on_data_source()` — `POST /{id}/query` | Validated, paginated query with the Redis cache (5-minute TTL, keyed by SQL + row cap) |
| `ConnectionManager.encrypt_password()` / `decrypt_password()` | Fernet encryption with `DB_ENCRYPTION_KEY`. In development, a key is generated once into `data/.encryption_key`; production refuses to start without one |
| `ConnectionManager.get_connection_url()` | Builds the async URL per type: `postgresql+asyncpg`, `mysql+aiomysql`, `mssql+aioodbc`, `oracle+oracledb`, `sqlite+aiosqlite` |
| `ConnectionManager.get_engine()` | One cached async engine per saved source (unsaved test sources are never cached) |
| `ConnectionManager.get_schema()` | Reflects tables via `inspect_schema_sync()`, caches success only, and drops a broken engine so the next try reconnects |

| Function | What it does |
|---|---|
| `ConnectionManager.invalidate_cache()` | Forgets engine and schema for one source |

`routes/sources.py` is the user-facing side: `GET /sources` lists the default database plus every active source (no host or credentials); `GET /sources/{id}/schema` returns tables, columns, primary keys and foreign keys (parsed by `_foreign_keys()` from the `col → table(col)` strings) and example questions from `build_suggestions()`; `POST /sources/{id}/refresh` (admin) re-reads the tables.

### 10.2 File analysis — `routes/file_analysis.py`, `services/file_session_store.py`

1. `POST /api/analyze-file` — `analyze_file()` reads the upload in chunks with `_read_limited()` and stops past **10 MB**. Only `.csv`, `.xlsx` and `.xls` are accepted.
2. `create_file_session()` reads up to **50 000 rows** with pandas, cleans column and table names (`_sanitize_col`, `_sanitize_table`, `_dedupe`), trims spaces in text cells, loads the data into an in-memory SQLite database and switches it to `PRAGMA query_only = ON`.
3. The response holds a `file_session_id`, the first 5 rows and an AI summary (`_llm_analysis()`, or `_fallback_analysis()` without AI) built from `_numeric_stats()`.
4. `POST /api/ask-file` — `_stream_file_answer()` checks the file belongs to this user, looks up where the searched words really occur (`_search_terms()`, `_find_values()`), asks the AI for SQL (`_generate_sql_for_file()`), runs the **same `validate_sql()`**, and streams `meta`, `token` and `done` events like the chat.
5. If the filter matches nothing, `_keyword_search_sql()` retries as a search for words that are in the file. The last 3 turns are kept for follow-up questions.
6. Sessions are private to the uploader, limited to 100 at once and dropped after **1 hour idle** (`_evict_expired_locked()`) or on restart. Results are capped at 5 000 rows.

### 10.3 Admin SQL console and confirmed changes — `routes/admin_sql.py`, `sql_safety/admin_writes.py`

Regular users are always read-only. An admin can run a change from the SQL console (Write mode) or by asking in chat — the chat only shows a yellow confirm card; the change runs when the admin clicks **Run change**.

`validate_admin_sql(sql, dialect, existing_tables, allow_full_table)` checks, in order:

1. No comments and only one statement (no extra `;`).
2. No `WRITE_BLOCKED_KEYWORDS` — GRANT, REVOKE, EXEC, CALL, ATTACH, PRAGMA, COPY, file functions, sleep functions — even for admins.
3. Parses with sqlglot; exactly one statement.
4. Kind (from `_kind_of()`) is SELECT, INSERT, UPDATE, DELETE, MERGE, CREATE, DROP, ALTER or TRUNCATE; CREATE/DROP only for TABLE, VIEW or INDEX.
5. No system schemas or tables.
6. Every table exists (except the one a CREATE makes, CTE names, or `DROP … IF EXISTS`).
7. UPDATE/DELETE without WHERE, TRUNCATE and DROP raise `AdminSQLError(needs_confirmation=True)` unless `allow_full_table` is set.

`execute_admin_sql()` — `POST /admin/sql/execute`:

| Step | What happens |
|---|---|
| Feature off? | 403 when `ADMIN_SQL_WRITES=false` |
| Validate | 400 for a forbidden statement; **409** for a whole-table change until `confirm_full_table: true` is sent (the browser asks a second time) |
| SELECT | Sent through the normal read-only path |
| Run | `_run_on_default()` or `_run_on_source()`: own transaction, committed on success, rolled back on error |
| On error | A plain message from `friendly_message()`; for a missing foreign-key value, `_existing_values()` suggests real ones |
| Log | `_audit()` writes an `[ADMIN WRITE]` row with status, rows affected and time |
| Refresh | Clears cached results; after CREATE/DROP/ALTER refreshes the table list and tells the UI (`schema_refreshed`) |

### 10.4 Admin panel and chat history — `routes/admin.py`, `routes/chat_history.py`

| Endpoint | Function | What it does |
|---|---|---|
| `GET /admin/stats` | `admin_stats()` | Counts users, active users, admins, total, successful and failed queries |
| `GET /admin/users` | `list_users()` | Paged user list ( `skip` , `limit` up to 200), newest first |
| `PATCH /admin/users/{id}` | `update_user()` | Promote or (de)activate. Admins cannot be demoted and cannot deactivate themselves |
| `GET /admin/audit-logs` | `list_audit_logs()` | Paged audit log, filter by `status` |
| `POST /chat/save` | `save_message()` | Saves one Q&A turn |
| `GET /chat/history` | `get_history()` | The user's last 30 turns (up to 200), newest first |
| `DELETE /chat/history` | `clear_history()` | Deletes the user's own history |

## 11. API reference

The backend exposes 39 endpoints from `main.py` and 11 route files; interactive docs are generated automatically at `http://localhost:8000/docs`. “Public” means no token; “User” means any logged-in account; “Admin” needs `role = admin`.

| Method | Path | Access | Purpose |
|---|---|---|---|
| GET | `/health` | Public | Default-database connectivity and version |
| POST | `/auth/register` | Public | Create account, email a code |
| POST | `/auth/send-otp` | Public | Resend a code |
| POST | `/auth/verify-otp` | Public | Verify code, log in |
| POST | `/auth/login` | Public | Email + password login |
| POST | `/auth/refresh` | Refresh cookie | New token pair |

| Method | Path | Access | Purpose |
|---|---|---|---|
| POST | `/auth/logout` | Refresh cookie | Revoke all sessions |
| POST | `/auth/forgot-password` | Public | Email a reset link |
| POST | `/auth/reset-password` | Reset token | Set a new password |
| GET | `/auth/me` | User | Current profile |
| POST | `/auth/google/token` | Public | Google popup login |
| GET | `/auth/google/redirect` | Public | Start Google redirect login |
| GET | `/auth/google/callback` | Public (Google) | Finish Google redirect login |
| POST | `/ask/stream` | User | Question → SSE stream (meta, token, confirm, done, error) |
| POST | `/ask` | User | Question → one JSON answer |
| POST | `/api/query` | User | Run typed read-only SQL (SQL console) |
| GET | `/sources` | User | Databases the user can pick |
| GET | `/sources/{id}/schema` | User | Tables, columns, keys, suggestions |
| POST | `/sources/{id}/refresh` | Admin | Re-read a database's tables |
| GET | `/api/schema` | User | Legacy: default database schema |
| POST | `/api/schema/refresh` | Admin | Legacy: refresh default schema |
| POST | `/api/analyze-file` | User | Upload CSV/Excel, get summary + session |
| POST | `/api/ask-file` | User | Question about an uploaded file (SSE) |
| POST | `/chat/save` | User | Save a Q&A turn |
| GET | `/chat/history` | User | Own history |
| DELETE | `/chat/history` | User | Clear own history |
| GET | `/admin/stats` | Admin | Usage numbers |
| GET | `/admin/users` | Admin | List users |
| PATCH | `/admin/users/{id}` | Admin | Change role / active state |

| Method | Path | Access | Purpose |
|---|---|---|---|
| GET | `/admin/audit-logs` | Admin | Audit log |
| POST | `/admin/sql/execute` | Admin | Run a confirmed change |
| POST | `/data-sources` | Admin | Add a database |
| GET | `/data-sources` | Admin | List databases |
| GET | `/data-sources/{id}` | Admin | One database |
| PATCH | `/data-sources/{id}` | Admin | Edit a database |
| DELETE | `/data-sources/{id}` | Admin | Remove a database |
| POST | `/data-sources/test` | Admin | Test connection details |
| GET | `/data-sources/{id}/schema` | Admin | Reflect tables with row counts |
| POST | `/data-sources/{id}/query` | Admin | Paged, cached query |

**Example — streaming answer.** Request: `POST /ask/stream` with `{"question": "Top 5 customers by spending", "session_id": "…", "data_source_id": null}`. Response frames, in order:

```
event: meta
data: {"sql_query": "SELECT c.name, SUM(o.total_amount) AS total_spent FROM
customers c JOIN orders o ON … LIMIT 5", "columns": ["name", "total_spent"],
"rows": [["Asha", 1820.5], …], "chart": {"type": "pie", …}, "timing_ms": 4}

event: token
data: {"token": "Asha is the top customer"}

event: done
data: {"explanation": "Asha is the top customer …"}
```

**Common error codes**: 400 invalid input or failed safety check; 401 missing, expired or revoked token; 403 not an admin or deactivated account; 404 not found; 409 whole-table change needs confirmation; 429 rate limit; 502 a data source cannot be reached or the AI call failed; 503 AI not configured.

# 12. Frontend

The frontend is a Next.js App Router app: each folder under `app/` is a page, shared layouts wrap them, and every call to the backend goes through `lib/api.ts`.

## 12.1 Pages and layouts

| Route | File | What it does |
|---|---|---|
| (all) | `app/layout.tsx` — `RootLayout` | Loads the Inter, Outfit and JetBrains Mono fonts and wraps the app in `MotionProvider` (respects reduced-motion) |
| `/` | `app/page.tsx` — `Root` | Redirects to `/dashboard` if a token exists, else `/auth` |
| `/auth` | `app/auth/page.tsx` — `AuthPage` | Tabs for login, register, code entry, forgot and reset password. `handleLogin`, `handleRegister`, `submitOTP`, `handleResend`, `handleForgot`, `handleResetPassword`; the 6 code boxes use `handleOtpInput`, `handleOtpKey` (backspace/arrows) and `handleOtpPaste`; `initGoogle` + `handleGoogleCredential` run Google's popup |
| `/auth/google-success` | `GoogleSuccessInner` | Reads the token from the URL fragment, removes it from the address bar, calls `saveSession()` and opens the dashboard |

| Route | File | What it does |
|---|---|---|
| `/dashboard/*` | `app/dashboard/layout.tsx` — `DashboardLayout` | Auth guard ( `ensureFreshToken()` , else back to login); loads databases, schema, health and history; listens for `schema-changed` and `history-changed` events; renders Navbar, Sidebar and the schema diagram; shares state through `DashContext` |
| | `app/dashboard/template.tsx` | Fades each dashboard page in on navigation |
| `/dashboard` | `app/dashboard/page.tsx` — `ChatPage` | The chat (12.2) |
| `/dashboard/sql` | `app/dashboard/sql/page.tsx` — `SqlConsolePage` | SQL console: `runQuery()` calls `/api/query` , or `/admin/sql/execute` in admin Write mode with a browser confirm (and a second one after a 409); `copySQL()` , `clearAll()` |
| `/admin` | `app/admin/page.tsx` — `AdminPage` | Tabs for Overview ( `StatCard` , animated `CountUp` ), Users ( `toggleActive` , `toggleRole` ), Audit log, Data sources ( `AddDataSourceModal` with `handleTest` / `handleSave` , `EditDataSourceModal` , `handleDeleteDS` , `handleToggleActive` ); `loadTab()` fetches each tab's data |

## 12.2 The chat — `app/dashboard/page.tsx`

| Function / component | What it does |
|---|---|
| `sendQuestion(question)` | Adds the user message and an empty "streaming" answer, picks `/ask/stream` (or `/api/ask-file` when a file is active), streams, then saves the turn with `saveChatMessage()` and fires `history-changed` |
| `streamSSE(url, body, onMeta, onToken, onDone, onError, onConfirm)` | Reads the response body with a stream reader, splits it into SSE frames on blank lines and calls the matching handler for `meta`, `token`, `done`, `error` and `confirm` |
| `handleFileUpload(file)`, `handleFileDrop(e)` | Posts the file to `/api/analyze-file` and switches the chat into file mode |
| `runChange(msgId, change)`, `cancelChange(msgId)` | Run or cancel an admin's pending change through `/admin/sql/execute`, with a second confirm for whole-table changes |
| `AnswerCard` | One answer: explanation, then tabs **Chart / Table / Joins / SQL**. Joins come from `parseJoins(meta.sql_query)` |
| `ChangeCard` | The yellow "Run change / Cancel" card |
| `FileCard` | Upload summary with a 5-row preview |
| `Thinking` | Rotating "Writing SQL… / Querying…" messages while waiting |
| `SqlCode`, `CopyButton` | Shows SQL with a copy button |

## 12.3 Components — `components/`

| Component | What it does |
|---|---|
| `Navbar` | Logo, database switcher (`onSourceChange`), health dot, New chat, links to SQL console and admin, user menu with `handleLogout()`. Loads the profile from `/auth/me` |

| Component | What it does |
|---|---|
| `Sidebar` | Tables and Recent tabs: search tables/columns, expand columns, “View schema diagram” button, admin refresh button; Recent re-fills the question box |
| `SchemaDiagram` | Full-screen ER diagram. `buildEdges()` uses declared foreign keys, then guesses `customer_id → customers`; `layout()` places referenced tables left and dependants right; pan, zoom, drag, search, fit; hover/click lights up a table’s links; details panel lists “Points to” and “Used by”; Framer Motion opening animation |
| `JoinVenn` (`JoinView`) | Draws each JOIN as two circles: dark middle = matching rows, light = rows kept without a match. Covers INNER, LEFT, RIGHT, FULL and CROSS, with a plain-English sentence per join |
| `ChartBlock` | Chart.js bar, line or pie chart from the result rows, using the `chart` hint from the backend |
| `DataTable`, `downloadCsv()` | Result table with a sticky header and right-aligned numbers (first 500 rows); CSV export of every row |
| `Avatar` | Profile picture: backend URL → unavatar.io → initials, moving on if an image fails |
| `Icon` | Small SVG icon set (`name` prop) |
| `LoadingScreen` | Full-page loader with message |
| `motion.tsx` | Shared animation presets (`spring`, `ease`, `fadeUp`, `stagger`, `collapse`) plus `Modal`, `TypingDots`, `Skeleton` |

## 12.4 `lib/api.ts` — the API client

| Function | What it does |
|---|---|
| `getToken()`, `getUser()` | Read the access token and user from `localStorage` |
| `isTokenValid()` | Decodes the JWT payload and checks expiry with a 30-second margin |

| Function | What it does |
|---|---|
| `saveSession(data)`, `clearSession()` | Store or remove token and user; `saveSession` fires `session-updated` so the Navbar updates |
| `refreshAccessToken()` | `POST /auth/refresh` with the cookie; only one refresh runs at a time, others share its promise |
| `ensureFreshToken()` | Refresh only if the token has expired |
| `authHeaders()` | `{Authorization: Bearer …}` |
| `authFetch(path, opts)` | Every authenticated call: refreshes an expiring token first and retries once on 401, else logs out |
| `apiPost(path, body)` | JSON POST for auth forms, turning error responses into readable messages |
| `errorText(body, fallback)` | Flattens FastAPI validation errors into one line |
| `fetchHealth()`, `fetchSources()`, `fetchSourceSchema()`, `refreshSourceSchema()` | Health and databases |
| `getSavedSourceId()`, `saveSourceId()` | Remember the chosen database in the browser |
| `fetchChatHistory()`, `saveChatMessage()`, `logout()`, `timeAgo()` | History, logout, “5m ago” labels |
| `fetchAdminStats()`, `fetchAdminUsers()`, `updateAdminUser()`, `fetchAuditLogs()` | Admin panel |
| `fetchDataSources()`, `createDataSource()`, `updateDataSource()`, `testDataSourceConnection()`, `deleteDataSource()` | Data-source management |

`lib/sqlJoins.ts` — `parseJoins(sql)` strips comments and string values, reads `FROM` and every `[INNER|LEFT|RIGHT|FULL|CROSS] JOIN … ON / USING`, resolves table aliases and returns `{kind, left, right, on, condition}` for each join.

**Styling.** `tailwind.config.ts` defines the teal brand colours, fonts, shadows and animations (`dash-flow`, `typing-dot` …); `app/globals.css` defines shared classes (`.card`, `.btn-primary`, `.input`, `.chip`, `.label`, `.skeleton`) and the Venn animations.

# 13. Security measures

Safety is layered: even if one check were bypassed, the next one still stops a harmful query.

| Threat | Protection | Code |
|---|---|---|
| AI or user writes harmful SQL (DROP, DELETE, data export) | 11-check validator on the syntax tree, not just text | `sql_safety/validator.py` |
| Query changes data anyway | Transaction always rolled back; file databases set to `query_only` | `db/executor.py`, `run_on_data_source()`, `file_session_store.py` |
| Huge or slow queries | Row cap ( `DB_MAX_ROWS` ), 5 s timeout on the default DB, 30 s on data sources | `_enforce_limit()`, `_set_timeout()` |
| Reading system tables or other schemas | Check 9 and the table whitelist (check 10) | `validator.py` |
| SQL injection through statement chaining or comments | Checks 1, 2 and 6 | `rules.py` |
| Stolen passwords | bcrypt, cost 12; strong-password rules | `core/security.py`, `schemas/auth.py` |
| Stolen or old sessions | 15-minute access tokens; httpOnly refresh cookie; `token_version` revokes all tokens on logout and reset | `core/security.py`, `auth.py` |
| Guessing codes or passwords | 5 tries per code; per-IP rate limits; constant-time comparison | `auth.py`, `core/rate_limit.py` |
| Discovering which emails exist | Same answer for known and unknown emails | `send_otp()`, `forgot_password()` |

| Threat | Protection | Code |
| --- | --- | --- |
| Reusing a reset link | Password fingerprint makes links single-use | `core/email_tokens.py` |
| Token leaking in logs | Google redirect puts the token in the URL fragment and the page removes it | `google_auth.py`, `google-success/page.tsx` |
| Data-source passwords on disk | Fernet encryption with `DB_ENCRYPTION_KEY`; never returned by the API | `connection_manager.py` |
| Users touching admin features | `require_admin` on every admin route; users never see hosts or credentials | `core/security.py`, `sources.py` |
| Accidental admin damage | Changes need confirmation; whole-table changes need a second one; GRANT, EXEC, ATTACH etc. blocked; every change audited | `admin_writes.py`, `admin_sql.py` |
| Reading another user's memory or file | Memory keyed by `user_id:session_id`; file sessions checked against the owner | `ask_stream.py`, `file_session_store.py` |
| Leaking internals in errors | Plain-English errors; driver details hidden in production | `core/friendly_errors.py`, `public_error()` |
| Insecure production config | Start-up refuses a weak `SECRET_KEY`, missing `DB_ENCRYPTION_KEY` or default admin password | `config.validate_production_settings()` |
| Cross-site requests | CORS limited to `CORS_ORIGINS` + `FRONTEND_URL`; cookie `SameSite=Lax` | `main.py` |

Development-only conveniences — showing the code or reset link when email fails, and logging in without Google credentials — are switched off whenever `APP_ENV` is not `development`.

## 14. Running, testing and deploying

Locally the project runs as two processes — the backend on port 8000 and the frontend on port 3000; with Docker, one command starts everything.

### Local setup

1. Backend: create a virtual environment, `pip install -r backend/requirements.txt`, copy `backend/.env.example` to `backend/.env` and set at least `SECRET_KEY`, `DATABASE_URL`, `TARGET_DB_URL` and `GROQ_API_KEY`.
2. Start it from `backend/`: `uvicorn app.main:app --reload --host 0.0.0.0 --port 8000`.
3. Frontend: create `frontend/.env.local` with `NEXT_PUBLIC_API_URL=http://localhost:8000`, then `npm install` and `npm run dev`.
4. Open `http://localhost:3000` and sign in as `admin` / `Admin1234` (change it straight away). API docs are at `http://localhost:8000/docs`.
5. Use `localhost` for both apps, not `127.0.0.1` for one of them, or the refresh cookie is not sent.

### Minimum `backend/.env`

```
APP_ENV=development
SECRET_KEY=any-long-random-string-at-least-32-characters
DATABASE_URL=sqlite+aiosqlite:///./data/app.db
TARGET_DB_URL=sqlite:///./data/analytics.db
GROQ_API_KEY=gsk_...
```

**Docker** — `docker compose up --build -d` starts four containers:

| Service | Image | Port | Notes |
|---|---|---|---|
| `postgres` | postgres:15-alpine | 5432 | App database, health-checked with `pg_isready` |
| `redis` | redis:7-alpine | 6379 | Optional cache, health-checked with `redis-cli ping` |
| `backend` | `backend/Dockerfile` (Python 3.13 + ODBC) | 8000 | Starts after both are healthy; `DATABASE_URL` and `REDIS_URL` point at the service names |
| `frontend` | `frontend/Dockerfile` | 3000 | `NEXT_PUBLIC_API_URL=http://localhost:8000` |

### Tests and checks

| Command | What it checks |
|---|---|
| `cd backend && python -m pytest -q` | 68 tests in 7 files: auth flow, SQL validator, sources, data sources, admin SQL, chat changes, file search. No network or API key needed |
| `cd frontend && npx tsc --noEmit && npm run lint` | TypeScript types and ESLint |
| `ruff format . && ruff check .` (backend), `npm run format` (frontend) | Code style |
| `check_errors.bat` | Runs all of the above on Windows and writes `check_results.txt` |

### Production checklist

- [ ] `APP_ENV=production`, a 32+ character `SECRET_KEY`, and `DB_ENCRYPTION_KEY` set and backed up
- [ ] Default admin password changed
- [ ] HTTPS in front of both apps; `COOKIE_SECURE=true`; real domains in `FRONTEND_URL`, `APP_URL`, `CORS_ORIGINS`
- [ ] SMTP configured so codes are emailed
- [ ] Read-only database accounts for `TARGET_DB_URL` and every data source (or a limited write account if admin changes are needed)
- [ ] PostgreSQL for `DATABASE_URL`, with backups
- [ ] Run without `--reload`

# 15. Limitations and future work

The current version covers the full question-to-answer loop safely; these are the known gaps and the next improvements.

| Limitation today | Effect | Possible improvement |
|---|---|---|
| Conversation memory and uploaded files live in server RAM | Lost on restart; does not scale across several servers | Store vectors in Redis or a vector database; keep files in object storage |
| Database changes use a small start-up migration list | Hard to manage many schema changes | Move to Alembic migrations |
| Answers depend on one hosted LLM (Groq) | No AI SQL if Groq is down or the key is missing (typed SQL still works) | Support a second provider or a local model as fallback |
| The whole schema is sent in each prompt | Very large databases can exceed the prompt size | Send only tables relevant to the question (schema retrieval) |
| Join view reads the SQL text | Old-style joins ( `FROM a, b WHERE a.x = b.y` ) are not drawn | Use sqlglot on the backend to return join details with the result |
| Rate limits are per IP | Users behind one proxy share a limit | Rate-limit per user as well |
| Access token in `localStorage` | Readable by scripts if an XSS bug appeared | Keep tokens only in httpOnly cookies |
| Charts are chosen by simple rules | A few results get a less suitable chart | Let users switch chart type; ask the LLM for a chart hint |
| Few frontend tests | UI regressions found by hand | Add component tests and end-to-end tests (Playwright) |